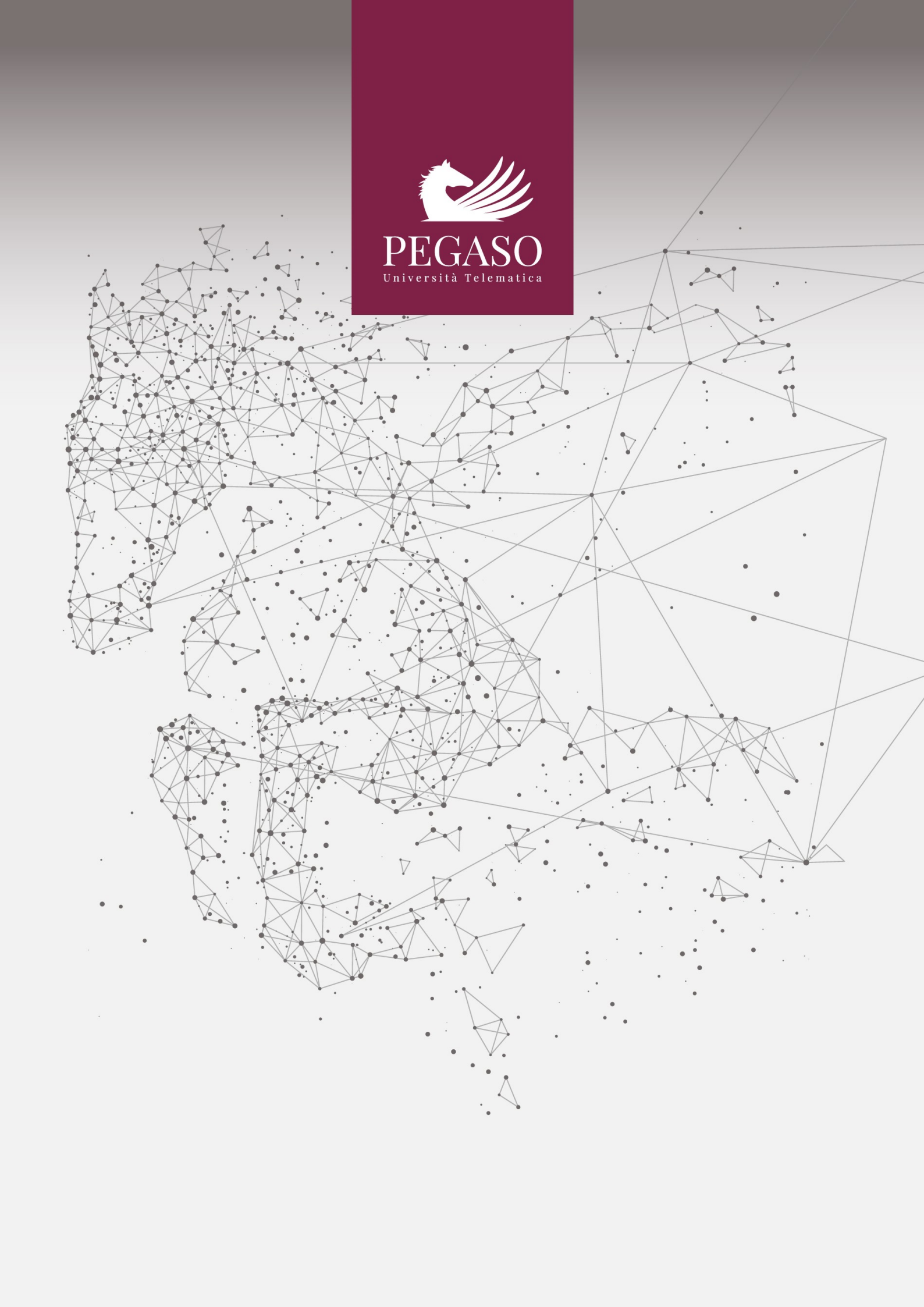

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. REACT 3
2. COMPONENTI 5
3. EVENTI 8
4. COMUNICAZIONE TRA COMPONENTI 12
BIBLIOGRAFIA 14
SITOGRAFIA 15

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. React

React è una libreria JavaScript open-source per la creazione di interfacce utente (UI). È stata sviluppata da Facebook e rilasciata nel 2013 come soluzione per semplificare lo sviluppo di applicazioni web complesse.

La peculiarità di React risiede nella sua architettura basata su componenti. In React, ogni elemento dell'interfaccia utente viene rappresentato come un componente riutilizzabile. Ciò significa che i componenti possono essere creati una volta e utilizzati ovunque nell'applicazione, riducendo la quantità di codice e semplificando la manutenzione dell'applicazione.

React utilizza una sintassi nota come "**JSX**" (JavaScript XML) che permette di definire la struttura dell'interfaccia utente in modo dichiarativo. JSX è una sintassi simile all'HTML, ma integrata con il JavaScript, il che rende il codice più leggibile e facile da mantenere.

Un'altra caratteristica di React è la gestione degli stati dei componenti. Gli stati consentono ai componenti di mantenere informazioni interne e di reagire ai cambiamenti nell'applicazione: quando lo stato di un componente cambia, React aggiorna automaticamente l'interfaccia utente per riflettere questo cambiamento.

Inoltre, React offre una vasta gamma di librerie e strumenti di supporto, come React Router per la gestione delle rotte dell'applicazione, Redux per la gestione dello stato globale dell'applicazione e Jest per i test automatizzati.

In sintesi, React è una libreria JavaScript potente e flessibile per la creazione di interfacce utente complesse e riutilizzabili. La sua architettura basata su componenti, la sintassi JSX e la gestione degli stati ne fanno una delle librerie più popolari per lo sviluppo di applicazioni web.

Per iniziare a utilizzare React, è necessario configurare un ambiente di sviluppo. Questo ambiente deve includere un'installazione di Node.js e di un package manager, come npm o yarn. Inoltre, è possibile utilizzare una varietà di editor di codice, come Visual Studio Code, Atom o Sublime Text, per sviluppare applicazioni React.

Per creare un nuovo progetto React, è possibile utilizzare il comando:

```
create-react-app
```

di npm. Questo comando crea una struttura di progetto preconfigurata con tutte le dipendenze necessarie per lo sviluppo di un'applicazione React.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, per creare un nuovo progetto React, è possibile eseguire il seguente comando:

```
npx create-react-app my-app
```

Questo comando crea un nuovo progetto React nella directory "my-app" e installa tutte le dipendenze necessarie.

Una volta creato il progetto, è possibile eseguirlo in modalità di sviluppo utilizzando il seguente comando:

```
cd my-app
npm start
```

Questo comando avvia l'applicazione React in un server di sviluppo locale, che consente di visualizzare l'applicazione nel browser e di effettuare modifiche al codice in tempo reale.

Una volta creato il progetto, è possibile creare il primo componente React. Un **componente** React è una funzione o una classe JavaScript che restituisce JSX attraverso una sintassi simile all'HTML, utilizzata per definire la struttura dell'interfaccia utente.

Ad esempio, un componente che restituisce un titolo potrebbe essere definito come segue:

```
function Titolo(props) {
  return <h1>{props.testo}</h1>;
}
```

In questo esempio, la funzione Titolo accetta un oggetto "props" come argomento e restituisce un elemento h1 che contiene il testo passato come proprietà "testo".

Una volta creato il componente, è possibile utilizzarlo in un'altra parte dell'applicazione. Ad esempio, per utilizzare il componente Titolo in un'altra parte dell'applicazione, è possibile includere il seguente codice JSX:

```
<Titolo testo="Benvenuti in React" />
```

In questo esempio, il componente Titolo viene utilizzato con la proprietà "testo" impostata su "Benvenuti in React".

La gestione dei **componenti** è uno dei concetti fondamentali di React: per utilizzare i componenti in modo efficace, è necessario capire come funzionano le proprietà e gli stati dei componenti. Le proprietà sono passate da un componente genitore a un componente figlio, mentre gli stati sono gestiti all'interno di un componente e possono essere aggiornati in risposta alle interazioni dell'utente o a eventi esterni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Componenti

In React, un componente è un'unità fondamentale dell'interfaccia utente. I componenti sono blocchi di codice riutilizzabili che rappresentano parti dell'interfaccia utente, come pulsanti, form e sezioni di layout.

I componenti possono essere definiti come classi JavaScript o come funzioni.

La struttura di un componente React dipende dal tipo di componente che si sta creando. Ad esempio, un componente definito come classe potrebbe avere il seguente aspetto:

```
class MyComponent extends React.Component {
  render() {
    return <h1>Hello, world!</h1>;
  }
}
```

In questo esempio, il componente MyComponent estende la classe base React.Component e definisce un metodo render() che restituisce un elemento h1 che contiene il testo "Hello, world!".

Un'altra caratteristica importante dei componenti React è la gestione delle props e degli state. Le props sono proprietà che vengono passate da un componente genitore a un componente figlio e possono essere utilizzate per personalizzare il comportamento e l'aspetto di un componente.

Ad esempio, il seguente componente accetta una prop "name" e la utilizza per restituire un saluto personalizzato:

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

In questo esempio, il componente Greeting utilizza la prop "name" per restituire un saluto personalizzato che include il nome passato come proprietà.

Gli stati, d'altra parte, sono gestiti all'interno di un componente e possono essere aggiornati in risposta alle interazioni dell'utente o a eventi esterni; gli stati consentono ai componenti di mantenere informazioni interne e di reagire ai cambiamenti nell'applicazione.

Ad esempio, il seguente componente gestisce uno stato interno che viene aggiornato quando l'utente fa clic su un pulsante:

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  }

  handleClick() {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>You clicked the button {this.state.count} times.</p>
        <button onClick={()=>this.handleClick()}>
           Click me</button>
      </div>
    );
  }
}
```

In questo esempio, il componente Counter definisce uno stato iniziale con il valore "count" impostato su 0. Quando l'utente fa clic sul pulsante, il metodo handleClick() aggiorna lo stato del componente utilizzando il metodo setState(). Il valore dello stato viene quindi visualizzato nell'interfaccia utente.

Infine, per creare un componente complesso in React, è possibile combinare diversi componenti più semplici in una struttura gerarchica. Ad esempio, il seguente componente complesso combina diversi componenti più semplici per creare un layout complesso:

```
function MyLayout(props) {
  return (
    <div>
      <header>
        <NavigationMenu />
      </header>
      <main>
        {props.children}
      </main>
      <footer>
        <ContactForm />
      </footer>
    </div>
  );
}
```

In questo esempio, il componente MyLayout definisce un layout complesso che include un'intestazione, un contenuto principale e un piè di pagina. Il componente NavigationMenu è un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

componente separato che viene utilizzato per visualizzare il menu di navigazione, mentre il componente ContactForm viene utilizzato per visualizzare un modulo di contatto nel piè di pagina.

Il componente MyLayout accetta una prop "children", che viene utilizzata per visualizzare il contenuto principale dell'applicazione. Ad esempio, per utilizzare il componente MyLayout con un contenuto principale personalizzato, è possibile includere il seguente codice JSX:

```
<MyLayout>
  <h1>Welcome to my app</h1>
  <p>This is the main content of my app.</p>
</MyLayout>
```

In questo esempio, il componente MyLayout viene utilizzato con un contenuto principale personalizzato che include un titolo e un paragrafo: il contenuto principale viene passato come figlio del componente MyLayout utilizzando la sintassi JSX.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Eventi

Gli eventi in React sono gestiti attraverso l'utilizzo di "handler" degli eventi, ovvero funzioni che vengono chiamate quando si verifica un evento specifico, come un clic su un pulsante.

In React, gli handler degli eventi vengono definiti utilizzando la sintassi di JSX e possono essere associati a vari tipi di eventi, come il clic su un pulsante o la pressione di un tasto sulla tastiera. Ad esempio, il seguente componente definisce un handler di eventi per gestire il clic su un pulsante:

```
class MyButton extends React.Component {
  handleClick() {
    console.log('Button clicked');
  }

  render() {
    return <button onClick={() => this.handleClick()}>
     Click me
    </button>;
  }
}
```

In questo esempio, il componente MyButton definisce un metodo handleClick() che viene chiamato quando l'utente fa clic sul pulsante. Il metodo handleClick() stampa un messaggio di log nella console del browser. L'handler di eventi viene quindi associato al pulsante utilizzando la sintassi onClick().

Oltre alla definizione degli handler di eventi all'interno delle classi, è possibile utilizzare anche gli "hooks" di React per definire gli handler di eventi all'interno di una funzione. Ad esempio, il seguente componente utilizza l'hook useState() per gestire uno stato interno che viene aggiornato quando l'utente fa clic su un pulsante:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>You clicked the button {count} times.</p>
      <button onClick={handleClick}>Click me</button>
    </div>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  );
}
```

In questo esempio, il componente Counter utilizza l'hook useState() per definire uno stato iniziale con il valore "count" impostato su 0. Quando l'utente fa clic sul pulsante, il metodo handleClick() utilizza il metodo setCount() per aggiornare lo stato del componente. Il valore dello stato viene quindi visualizzato nell'interfaccia utente.

Infine, il rendering condizionale è un aspetto importante della creazione di componenti React dinamici. Il rendering condizionale consente di mostrare o nascondere parti dell'interfaccia utente in base alle condizioni del componente.

Ad esempio, il seguente componente utilizza una condizione per mostrare un messaggio di errore se il valore di una prop è nullo:

```
function ErrorMessage(props) {
  if (!props.message) {
    return null;
  }

  return <p>{props.message}</p>;
}
```

In questo esempio, il componente ErrorMessage utilizza la condizione if per verificare se la prop "message" è nulla: se la prop è nulla, il componente restituisce null, altrimenti restituisce un elemento p che contiene il testo della prop.

Il rendering condizionale può essere utilizzato anche per creare componenti che cambiano il loro comportamento in base allo stato o alle props. Ad esempio, il seguente componente utilizza una condizione per mostrare un messaggio di benvenuto personalizzato in base al nome passato come prop:

```
function WelcomeMessage(props) {
  if (props.name) {
    return <p>Welcome, {props.name}!</p>;
  }

  return <p>Welcome, stranger!</p>;
}
```

In questo esempio, il componente WelcomeMessage utilizza la condizione if per verificare se la prop "name" è presente: se la prop è presente, il componente restituisce un messaggio di benvenuto personalizzato che include il nome passato come proprietà, altrimenti restituisce un messaggio di benvenuto generico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il seguente componente utilizza l'istruzione if/else per mostrare un messaggio di errore se la prop "errorMessage" è presente:

```
function Error(props) {
  if (props.errorMessage) {
    return <p>{props.errorMessage}</p>;
  } else {
    return null;
  }
}
```

Se la prop è presente, il componente restituisce un elemento p che contiene il messaggio di errore, altrimenti restituisce null.

È possibile utilizzare anche il ternario per creare il rendering condizionale in React; ad esempio, il seguente componente utilizza il ternario per mostrare un messaggio di benvenuto personalizzato in base alla presenza della prop "name":

```
function Welcome(props) {
  return (
    <div>
      {props.name ? (
        <h1>Welcome, {props.name}!</h1>
      ) : (
        <h1>Welcome, guest!</h1>
      )}
    </div>
  );
}
```

In questo esempio, il componente Welcome utilizza il ternario per verificare se la prop "name" è presente. Se la prop è presente, il componente restituisce un elemento h1 che contiene il messaggio di benvenuto personalizzato, altrimenti restituisce un elemento h1 che contiene un messaggio di benvenuto generico.

Inoltre, è possibile utilizzare le strutture di controllo, come i cicli for o le istruzioni switch, per il rendering condizionale in React. Ad esempio, il seguente componente utilizza un ciclo for per creare una lista di elementi in base alle props passate:

```
function List(props) {
  const items = [];

  for (let i = 0; i < props.count; i++) {
    items.push(<li key={i}>Item {i}</li>);
  }
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  return <ul>{items}</ul>;
}
```

In questo esempio, il componente List utilizza un ciclo for per creare una lista di elementi li in base alla prop "count". Ogni elemento li viene aggiunto a un array "items", che viene poi restituito come figlio dell'elemento ul.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Comunicazione tra componenti

La comunicazione tra componenti è un aspetto importante della creazione di applicazioni React complesse: i componenti React possono comunicare tra loro passando props da un componente genitore a un componente figlio.

Ad esempio, il seguente componente utilizza una prop "message" per passare un messaggio da un componente genitore a un componente figlio:

```
function Parent() {
  return <Child message="Hello, world!" />;
}

function Child(props) {
  return <p>{props.message}</p>;
}
```

In questo esempio, il componente Parent passa la prop "message" al componente Child. Il componente Child utilizza poi la prop "message" per restituire un elemento p che contiene il messaggio.

La comunicazione tra componenti in React può essere gestita in diversi modi, tra cui l'utilizzo dei callbacks e dei pattern di sollevamento dello stato.

L'utilizzo dei callbacks consente ai componenti figli di comunicare con i componenti genitori passando loro delle funzioni di callback come props. Ad esempio, il seguente componente utilizza un callback per notificare al componente genitore quando l'utente ha fatto clic su un pulsante:

```
function Child(props) {
  function handleClick() {
    props.onButtonClick();
  }

  return <button onClick={handleClick}>Click me</button>;
}

function Parent() {
  function handleButtonClick() {
    console.log('Button clicked');
  }

  return <Child onButtonClick={handleButtonClick} />;
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando l'utente fa clic sul pulsante, il componente Child chiama il callback "onButtonClick", che è definito nel componente genitore.

Il pattern di sollevamento dello stato, invece, consente ai componenti di comunicare tra loro sollevando lo stato a un livello superiore nella gerarchia dei componenti. Ad esempio, il seguente componente utilizza il pattern di sollevamento dello stato per aggiornare lo stato di un componente padre quando un componente figlio viene modificato:

```
function Child(props) {
  function handleChange(event) {
    props.onChange(event.target.value);
  }

  return <input type="text"
     value={props.value} onChange={handleChange} />;
}

class Parent extends React.Component {
  constructor(props) {
    super(props);
    this.state = { value: '' };
    this.handleChange = this.handleChange.bind(this);
  }

  handleChange(value) {
    this.setState({ value: value });
  }

  render() {
    return <Child value={this.state.value}
      onChange={this.handleChange} />;
  }
}
```

In questo esempio, il componente Child utilizza la prop "onChange" per passare un callback dal componente genitore al componente figlio: quando l'utente modifica il valore dell'input, il componente Child chiama il callback "onChange" con il nuovo valore. Il componente Parent utilizza poi il valore aggiornato per aggiornare lo stato del componente.

Inoltre, esistono diverse librerie per la gestione della comunicazione tra componenti in React, come Redux e MobX. Queste librerie offrono una soluzione più avanzata per la gestione dello stato e della comunicazione tra componenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org
- REACT: https://it.legacy.reactjs.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).